

HPC vs (Big) Data

Jascha Schewtschenko

Royal Observatory of Edinburgh, University of Edinburgh

May 16, 2025



Outline

- 1 Data
- 2 (Semi-)Structured Data / Databases
 - Database types / Data models
- 3 Unstructured Data / Filesystems
 - Distributed Data Storage/Filesystem
- 4 Hadoop
 - MapReduce



Data



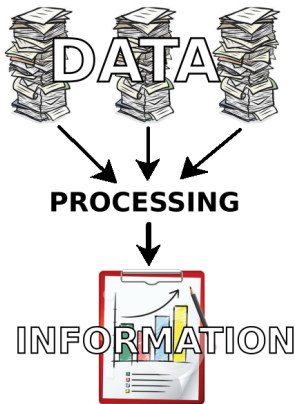
Data vs Information



Data 'raw' data / plain facts e.g. a text file, employee database, measurements, . . .



Data vs Information



Data 'raw' data / plain facts e.g. a text file, employee database, measurements, ...

Information 'processed'/meaningful data **in a** given context (e.g. word count in textfile, avg/variance of employees' salary, trend in the measurements)

Structured vs Unstructured Data

Structured Data



0.103	0.176	0.387	0.300	0.379
0.333	0.384	0.564	0.587	0.857
0.421	0.309	0.654	0.729	0.228
0.266	0.750	1.056	0.936	0.911
0.225	0.326	0.643	0.337	0.721
0.187	0.586	0.529	0.340	0.829
0.153	0.485	0.560	0.428	0.628

Structured vs Unstructured Data

Structured Data



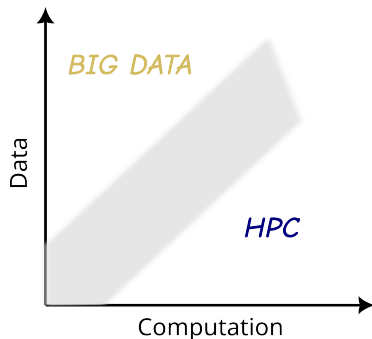
0.103	0.176	0.387	0.300	0.379
0.333	0.384	0.564	0.587	0.857
0.421	0.309	0.654	0.729	0.228
0.266	0.750	1.056	0.936	0.911
0.225	0.326	0.643	0.337	0.721
0.187	0.586	0.529	0.340	0.829
0.153	0.485	0.560	0.428	0.628

Unstructured Data



HPC vs “Big Data”

- High Performance Computing and Big Data are two important approaches to scientific computing
- HPC typically deals with smaller, highly structured data sets and huge amounts of computation
- “Big Data” deals with gigantic, usually unstructured data sets and focuses on the I/O bottlenecks
- with “Big Data” becoming more computational-heavy (e.g. for Data Science / Machine Learning) and HPC more I/O/data-heavy (e.g. processing of big observations), there is a big gray zone where techniques from both fields are relevant



(Semi-)Structured Data / Databases



Databases

- Define the logical structure on data and relations (data model)



Databases

- Define the logical structure on data and relations (data model)
- provides a mechanism for storage and retrieval of data



Databases

- Define the logical structure on data and relations (data model)
- provides a mechanism for storage and retrieval of data
- Database-management system (DBMS) interacts with end-users, other applications, and the database itself to capture and analyse data



Databases

- Define the logical structure on data and relations (data model)
- provides a mechanism for storage and retrieval of data
- Database-management system (DBMS) interacts with end-users, other applications, and the database itself to capture and analyse data
- A general-purpose DBMS allows the definition, creation, querying, update, and administration of databases.

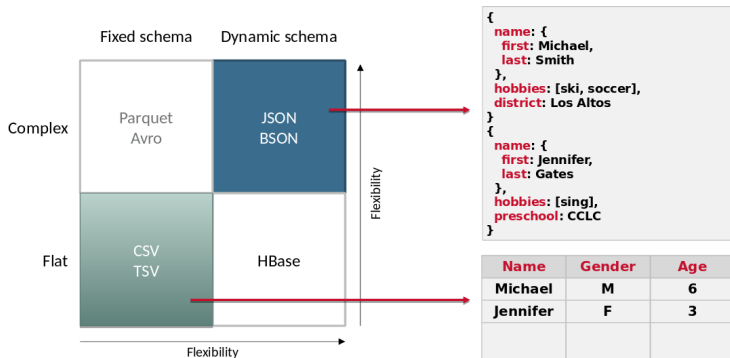


Databases

- Define the logical structure on data and relations (data model)
- provides a mechanism for storage and retrieval of data
- Database-management system (DBMS) interacts with end-users, other applications, and the database itself to capture and analyse data
- A general-purpose DBMS allows the definition, creation, querying, update, and administration of databases.
- A database is **NOT** generally portable across different DBMSs, but different DBMSs can interoperate by using standards (e.g. SQL, ODBC)



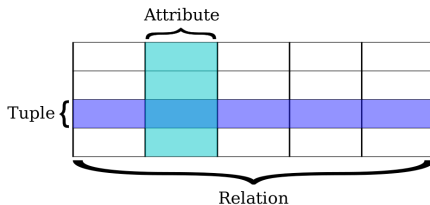
Data(base) types/models



- Data usually has an intrinsic complexity (relations & variability of schema)
- Type of data and how it will be used determines which data model to use

Relational Databases

- all data is represented in terms of tuples, grouped into relations (aka tables)



- virtually all relational database systems use SQL ($\rightarrow$ SQL databases)

Non-Relational Databases

- Anything **BUT** relational/SQL databases (→ NoSQL databases)
- NoSQL databases are often very fast, do not require fixed table schemas, avoid join operations by storing denormalized data, and are designed to scale horizontally.
- Classes of NoSQL DBs and examples:

Column Store stores data as columns rather than rows (e.g. Amazon DynamoDB, Bigtable, Cassandra, Druid, HBase, Hypertable)

Key-Value least complex NoSQL option; data stored in schema-less way that consists of indexed keys and values (e.g. Aerospike, Apache Ignite, ArangoDB, Couchbase, InfinityDB, Oracle NoSQL DB, OrientDB, Redis, Riak)

Document Store key-value concept with added complexity; each document has its own data & unique key (e.g. ArangoDB, BaseX, Clusterpoint, Couchbase, CouchDB, DocumentDB)

Graph designed for data whose relations are well represented as a graph (e.g. AllegroGraph, ArangoDB, InfiniteGraph, Apache Giraph, MarkLogic)



Relational vs Non-Relational Databases

Each class of DB has certain advantages/disadvantages:

Data model ↕	Performance ↕	Scalability ↕	Flexibility ↕	Complexity ↕
Key-value store	high	high	high	none
Column-oriented store	high	high	moderate	low
Document-oriented store	high	variable (high)	high	low
Graph database	variable	variable	high	high
Relational database	variable	variable	low	moderate

Performance rate at which a database management system (DBMS) supplies information to users

Scalability capability of a system to handle a growing amount of work in the same elapsed time when processing power is expanded to accommodate growth

Flexibility ability to deal with changing/variety of dataset types

Complexity complexity of DBMS/queries



Clustered/Distributed Filesystems



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):
 - Access transparency files while distributed can be accessed in the same way as local files are accessed



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):
 - Access transparency** files while distributed can be accessed in the same way as local files are accessed
 - Location transparency** existence of consistent name space encompassing local as well as remote files



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):
 - Access transparency** files while distributed can be accessed in the same way as local files are accessed
 - Location transparency** existence of consistent name space encompassing local as well as remote files
 - Concurrency transparency** all clients have the same view of the state of the file system



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):
 - Access transparency** files while distributed can be accessed in the same way as local files are accessed
 - Location transparency** existence of consistent name space encompassing local as well as remote files
 - Concurrency transparency** all clients have the same view of the state of the file system
 - Heterogeneity** file service should be provided across different hardware and operating system platforms



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):

Access transparency files while distributed can be accessed in the same way as local files are accessed

Location transparency existence of consistent name space encompassing local as well as remote files

Concurrency transparency all clients have the same view of the state of the file system

Heterogeneity file service should be provided across different hardware and operating system platforms

Scalability file system should work well indep. of environment size



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):

Access transparency files while distributed can be accessed in the same way as local files are accessed

Location transparency existence of consistent name space encompassing local as well as remote files

Concurrency transparency all clients have the same view of the state of the file system

Heterogeneity file service should be provided across different hardware and operating system platforms

Scalability file system should work well indep. of environment size

Replication transparency mask that files may be replicated across multiple servers (to support scalability/redundancy)



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):

Access transparency files while distributed can be accessed in the same way as local files are accessed

Location transparency existence of consistent name space encompassing local as well as remote files

Concurrency transparency all clients have the same view of the state of the file system

Heterogeneity file service should be provided across different hardware and operating system platforms

Scalability file system should work well indep. of environment size

Replication transparency mask that files may be replicated across multiple servers (to support scalability/redundancy)



Clustered/Distributed File systems

- File system is *schemaless*, i.e. allows to store unstructured data
- shared-disk (cf. SAN) vs distributed file systems
- Goals (among others):

Access transparency files while distributed can be accessed in the same way as local files are accessed

Location transparency existence of consistent name space encompassing local as well as remote files

Concurrency transparency all clients have the same view of the state of the file system

Heterogeneity file service should be provided across different hardware and operating system platforms

Scalability file system should work well indep. of environment size

Replication transparency mask that files may be replicated across multiple servers (to support scalability/redundancy)

- *parallel file systems* are a type of clustered file system that spread data across multiple storage nodes, usually for redundancy or performance



Clustered/Distributed File systems : LUSTRE

- initially developed as a research project (Braam 1999)



Clustered/Distributed File systems : LUSTRE

- initially developed as a research project (Braam 1999)
- first developed in 2003 at MCR Linux Cluster at the Lawrence Livermore National Laboratory (Top3)

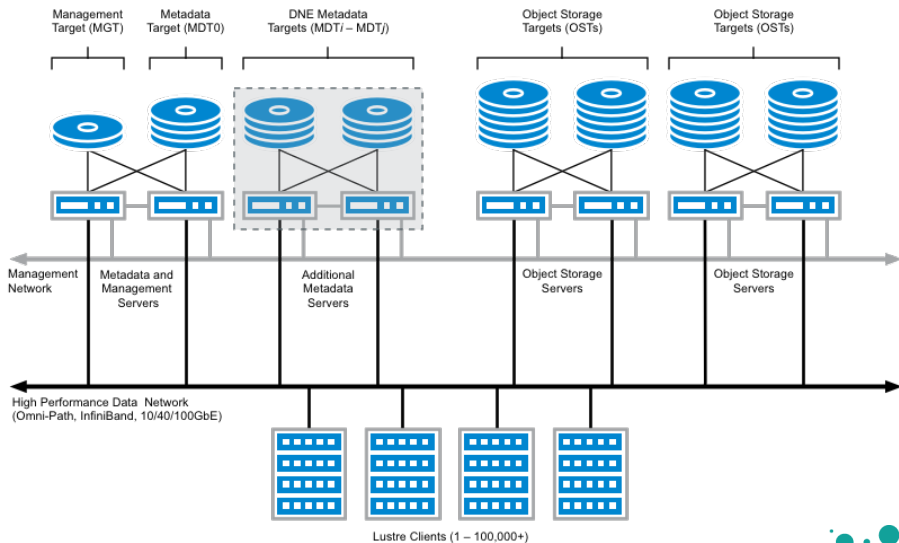


Clustered/Distributed File systems : LUSTRE

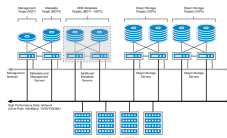
- initially developed as a research project (Braam 1999)
- first developed in 2003 at MCR Linux Cluster at the Lawrence Livermore National Laboratory (Top3)
- now consistently used on majority of top supercomputers for last two decades



Clustered/Distributed File systems : LUSTRE

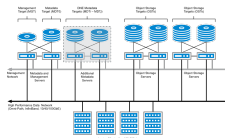


Clustered/Distributed File systems : LUSTRE



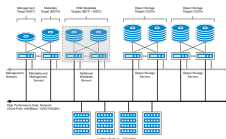
- building blocks of a Lustre file system are
Metadata Servers (MDS) manages all namespace operations for a Lustre file system, such as directories (folders) and filenames; data stored in MDTs

Clustered/Distributed File systems : LUSTRE



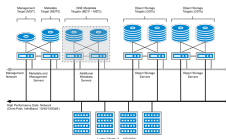
- building blocks of a Lustre file system are
 - Metadata Servers (MDS)** manages all namespace operations for a Lustre file system, such as directories (folders) and filenames; data stored in MDTs
 - Object Storage Server (OSS)** OSS nodes manage the bulk storage for the contents of files in a Lustre file system; data stored in OSTs

Clustered/Distributed File systems : LUSTRE



- building blocks of a Lustre file system are
 - Metadata Servers (MDS)** manages all namespace operations for a Lustre file system, such as directories (folders) and filenames; data stored in MDTs
 - Object Storage Server (OSS)** OSS nodes manage the bulk storage for the contents of files in a Lustre file system; data stored in OSTs
 - Management Server (MGS)** stores configuration information for all the Lustre file systems in a cluster in the MDTs

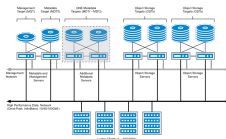
Clustered/Distributed File systems : LUSTRE



- building blocks of a Lustre file system are
 - Metadata Servers (MDS)** manages all namespace operations for a Lustre file system, such as directories (folders) and filenames; data stored in MDTs
 - Object Storage Server (OSS)** OSS nodes manage the bulk storage for the contents of files in a Lustre file system; data stored in OSTs
 - Management Server (MGS)** stores configuration information for all the Lustre file systems in a cluster in the MDTs
 - Lustre Networking (LNet)** high-speed data network protocol that clients use to access the file system

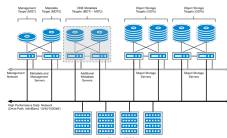


Clustered/Distributed File systems : LUSTRE



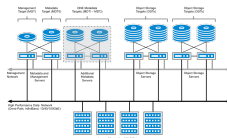
- building blocks of a Lustre file system are
 - Metadata Servers (MDS)** manages all namespace operations for a Lustre file system, such as directories (folders) and filenames; data stored in MDTs
 - Object Storage Server (OSS)** OSS nodes manage the bulk storage for the contents of files in a Lustre file system; data stored in OSTs
 - Management Server (MGS)** stores configuration information for all the Lustre file systems in a cluster in the MDTs
 - Lustre Networking (LNet)** high-speed data network protocol that clients use to access the file system
 - Clients** represented as a file system mount point on a host and presents applications with a unified namespace for all of

Clustered/Distributed File systems : LUSTRE



- targets are disk racks (either smart (RAID) or just 'just a bunch of discs' (JBOD)); each OSS manages multiple OSTs usually OSS work in teams to be connected to same OSTs for redundancy

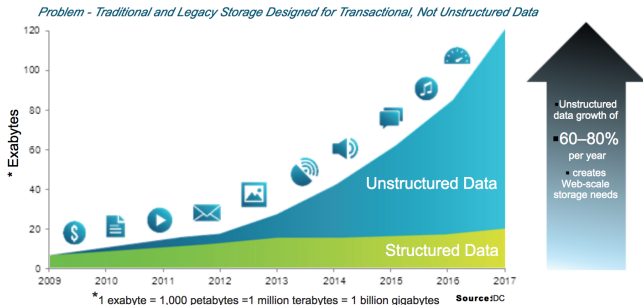
Clustered/Distributed File systems : LUSTRE



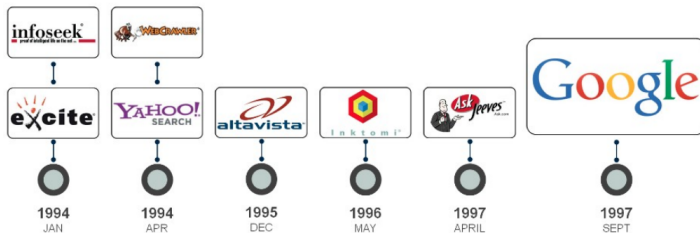
- targets are disk racks (either smart (RAID) or just 'just a bunch of discs' (JBOD)); each OSS manages multiple OSTs usually OSS work in teams to be connected to same OSTs for redundancy
- usually OSS work in teams to be connected to same OSTs for redundancy, managing their share, but jumping in to take over if teammate fails (same for MGS/MDS)

Data Growth

Data Growth



Search Engines





- *Doug Cutting* authored highly successful OSS information retrieval software library: Lucene
- Planned to use it on internet; Developed Webcrawler with HTML parser for Lucene with *Mike Carafella* (written completely in Java): Nutch
- proof-of-concept deployed on a single machine in 2003 (single-core processor, 1GB of RAM, RAID level 1 on eight hard drives, amounting to 1TB)
- worked on 100 million webpages
- **Challenge:** To scale it up i.e. they needed distributed data storage & processing



Clustered/Distributed File systems : GFS

The Google File System

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung

Google*

ABSTRACT

We have designed and implemented the Google File System, a scalable distributed file system for large distributed data-intensive applications. It provides fault tolerance while running on inexpensive commodity hardware, and it delivers high aggregate performance to a large number of clients.

While sharing many of the same goals as previous distributed file systems, our design has been driven by observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure from some earlier file system assumptions. This has led us to reexamine traditional choices and explore radically different design points.

The file system has successfully met our storage needs. It is widely deployed within Google as the storage platform for the generation and processing of data used by our service as well as research and development efforts that require large data sets. The largest cluster to date provides hundreds of terabytes of storage across thousands of disks on

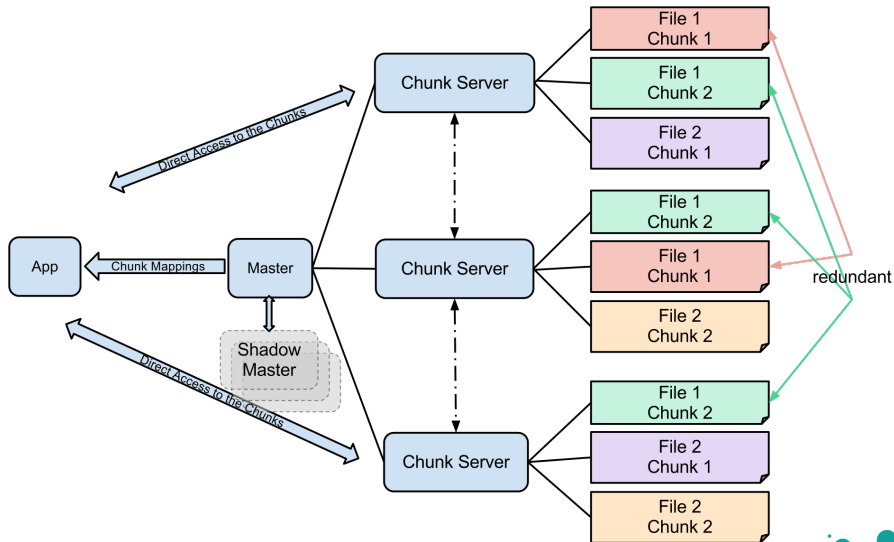
1. INTRODUCTION

We have designed and implemented the Google File System (GFS) to meet the rapidly growing demands of Google's data processing needs. GFS shares many of the same goals as previous distributed file systems such as performance, scalability, reliability, and availability. However, its design has been driven by key observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure from some earlier file system design assumptions. We have reexamined traditional choices and explored radically different points in the design space.

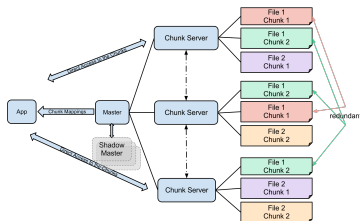
First, component failures are the norm rather than the exception. The file system consists of hundreds or even thousands of storage machines built from inexpensive commodity parts and is accessed by a comparable number of client machines. The quantity and quality of the components virtually guarantee that some are not functional at any given time and some will not recover from their cur-



Clustered/Distributed File systems : GFS

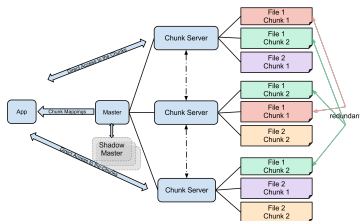


Clustered/Distributed File systems : GFS



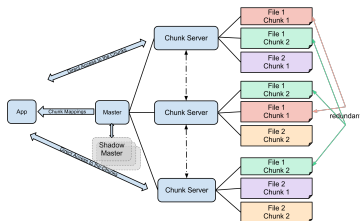
- files divided into chunks/blocks (of about 128MB by default)

Clustered/Distributed File systems : GFS



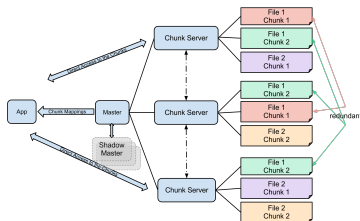
- files divided into chunks/blocks (of about 128MB by default)
- building blocks of a Hadoop file system are

Clustered/Distributed File systems : GFS



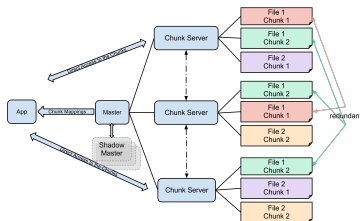
- files divided into chunks/blocks (of about 128MB by default)
- building blocks of a Hadoop file system are
NameNode (Master) high-reliability system maintaining the file system/meta data & location of blocks (with shadow NameNodes/CheckpointNodes for recovery)

Clustered/Distributed File systems : GFS



- files divided into chunks/blocks (of about 128MB by default)
- building blocks of a Hadoop file system are
 - NameNode (Master)** high-reliability system maintaining the file system/meta data & location of blocks (with shadow NameNodes/CheckpointNodes for recovery)
 - DataNode (ChunkServer)** commodity system storing the data blocks and processing them

Clustered/Distributed File systems : GFS



- files divided into chunks/blocks (of about 128MB by default)
- building blocks of a Hadoop file system are
 - NameNode (Master)** high-reliability system maintaining the file system/meta data & location of blocks (with shadow NameNodes/CheckpointNodes for recovery)
 - DataNode (ChunkServer)** commodity system storing the data blocks and processing them
- rack-awareness for reliability - two chunks usually stored on DataNode in same rack, third off-rack

Distributed/Parallel data processing : MapReduce

MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

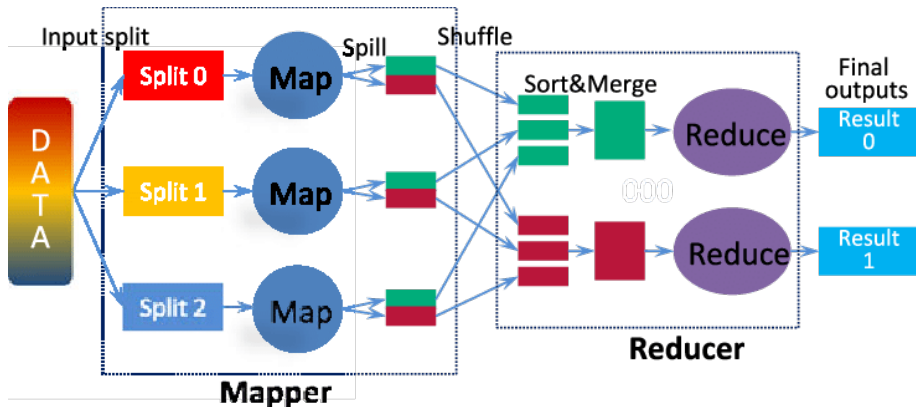
Programs written in this functional style are automati-

given day, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

As a reaction to this complexity, we designed a new abstraction that allows us to express the simple computa-



MapReduce



MapReduce

MapReduce solves:



MapReduce

MapReduce solves:

Parallelization how to parallelize the computation



MapReduce

MapReduce solves:

Parallelization how to parallelize the computation

Distribution how to distribute the data



MapReduce

MapReduce solves:

Parallelization how to parallelize the computation

Distribution how to distribute the data

Fault-tolerance how to handle program failure



MapReduce: Example

- *Problem:* Determine median value of all 'numbered' cards for each colour



MapReduce: Example



- *Problem:* Determine median value of all 'numbered' cards for each colour
- *Mapper:*
 - ▶ Run image recognition on (local) stack of cards to determine colour and value number on card)
 - ▶ Discard 'invalid' cards & create colour-value key-value pairs for valid ones

MapReduce: Example



- *Problem:* Determine median value of all 'numbered' cards for each colour
- *Mapper:*
 - ▶ Run image recognition on (local) stack of cards to determine colour and value number on card)
 - ▶ Discard 'invalid' cards & create colour-value key-value pairs for valid ones
- *Combiner/Shuffle:*
 - ▶ Combine key-value pairs by key (i.e. colour)
 - ▶ Data gets redistributed to ensure that all key-value pairs of the same type end up on same node

MapReduce: Example



- *Problem:* Determine median value of all 'numbered' cards for each colour
- *Mapper:*
 - ▶ Run image recognition on (local) stack of cards to determine colour and value number on card)
 - ▶ Discard 'invalid' cards & create colour-value key-value pairs for valid ones
- *Combiner/Shuffle:*
 - ▶ Combine key-value pairs by key (i.e. colour)
 - ▶ Data gets redistributed to ensure that all key-value pairs of the same type end up on same node
- *Mapper:*
 - ▶ Calculate Median on (complete) set of cards for the assigned colour

MapReduce: Example



- *Problem:* Determine median value of all 'numbered' cards for each colour
- *Mapper:*
 - ▶ Run image recognition on (local) stack of cards to determine colour and value number on card)
 - ▶ Discard 'invalid' cards & create colour-value key-value pairs for valid ones
- *Combiner/Shuffle:*
 - ▶ Combine key-value pairs by key (i.e. colour)
 - ▶ Data gets redistributed to ensure that all key-value pairs of the same type end up on same node
- *Mapper:*
 - ▶ Calculate Median on (complete) set of cards for the assigned colour

Hadoop



Map/Reduce

Map/Reduce

BigTable

HBase

GFS

HDFS

Hadoop: Code Example

- Implement Mapper and Reducer:

```
//Mapper class
public static class E_EMapper extends MapReduceBase implements
Mapper<LongWritable, /*Input key Type */
Text, /*Input value Type*/
Text, /*Output key Type*/
IntWritable> /*Output value Type*/
{
    //Map function
    public void map(LongWritable key, Text value,
OutputCollector<Text, IntWritable> output,
Reporter reporter) throws IOException {
        String line = value.toString();
        String lasttoken = null;
        StringTokenizer s = new StringTokenizer(line, "\\t");
        String year = s.nextToken();

        while(s.hasMoreTokens()) {
            lasttoken = s.nextToken();
        }
        int avgprice = Integer.parseInt(lasttoken);
        output.collect(new Text(year), new IntWritable(avgprice));
    }
}

//Reducer class
public static class E_EReduce extends MapReduceBase implements Reducer< T
//Reduce function
public void reduce( Text key, Iterator <IntWritable> values,
OutputCollector<Text, IntWritable> output, Reporter reporter) throws I
    int maxavg = 30;
    int val = Integer.MIN_VALUE;

    while (values.hasNext()) {
        if((val = values.next().get())>maxavg) {
            output.collect(key, new IntWritable(val));
        }
    }
}
```

- Use them to build Hadoop pipeline:

```
//Main function
public static void main(String args[])throws Exception {
    JobConf conf = new JobConf(ProcessUnits.class);

    conf.setJobName("max_electricityunits");
    conf.setOutputKeyClass(Text.class);
    conf.setOutputValueClass(IntWritable.class);
    conf.setMapperClass(E_EMapper.class);
    conf.setCombinerClass(E_EReduce.class);
    conf.setReducerClass(E_EReduce.class);
    conf.setInputFormat(TextInputFormat.class);
    conf.setOutputFormat(TextOutputFormat.class);

    FileInputFormat.setInputPaths(conf, new Path(args[0]));
    FileOutputFormat.setOutputPath(conf, new Path(args[1]));

    JobClient.runJob(conf);
}
```

